TP : Transactions

Récupérer sur moodle le fichier BDPiece.sql, et enregitrer le dans le répertoire Mes_projets_psql exécuter le fichier : \i BDPiece.sql.

À partir de l'exercice 2, il faut se connecter en mode multi-utilisateurs (comme expliqué ci-dessous).

Connexion en multi-utilisateurs

1. Ouvrir un terminal puis :

- Se connecter sur le conteneur en mode Mult-utilisateurs : ***_postgresqlDev_multi***
- Se connecter à la base postgesql : ***psql postgres***
- Exécuter le fichier \i BDPiece.sql sur cette base (pour créer les tables avec les n-uplets)

2. Ouvrir un autre terminal (soit sur votre machine ou celle du voisin)

- pour travailler seul : ***psql -h localhost -U postgres***

- pour travailler avec son voisin : ***psql -h <ad_ip> -U postgres***

<ad_ip> est l'adresse IP de la machine (par exemple, 172.20.40.226). On l'obtient avec la commande ***ip addr.*** Dans la réponse, il faut aller voir l'inet du deuxième item (en général 2: enp0s31f6).

EXERCICE 1

On considère la séquence de requête SQL suivante :

```
-----
begin transaction;
insert into representation values (107,'Le Quai','23-01-2021','début de soirée');
insert into representation values (107,'Le Quai','23-01-2021','matin');
commit;
select * from representation where numero='107';
-----
```

1. Donner le nombre et les transactions contenues dans cette séquence.
2. Expliquer requête par requête l'exécution de ce code.

On considère la nouvelle séquence de requête SQL suivante :

```
-----
begin transaction;
insert into representation values (107,'Le Quai','23-01-2021','début de soirée');
insert into representation values (107,'Le Quai','23-01-2021','matin');
select * from representation where numero='107';
commit;
-----
```

1. Donner le résultat de la requête SELECT et expliquer.

On considère la nouvelle séquence de requête SQL suivante :

```
-----
begin transaction;
insert into representation values (107,'Le Quai','23-01-2021','début de soirée');
select * from representation where numero='107';
insert into representation values (107,'Le Quai','23-01-2021','matin');
commit;
select * from representation where numero='107';
-----
```

1. Donner le résultat des deux requêtes SELECT et expliquer.

EXERCICE 2

À partir de cet exercice, il faut se connecter en mode multi-utilisateurs.

1. Faites des modifications (pas de create ou de drop) dans une des sessions et voyez si les modifications sont connues de l'autre session. Exemple : changer le nom complet d'un auteur dans une session et afficher le contenu de cette table dans les deux sessions.
2. Commencer une transaction dans une session avec la requête BEGIN TRANSACTION.
3. Faites des modifications (pas de create ou de drop) dans cette session et voyez si les modifications sont connues de l'autre session.
4. Faites un COMMIT des modifications et voyez si les modifications sont connues de l'autre session.
5. Sans utiliser la marque de début de transaction, modifiez le nom complet d'un même auteur dans les deux sessions (avec des valeurs différentes). Que se passe-t-il ?
6. Commencer une transaction dans chacune des sessions avec la requête BEGIN TRANSACTION.
7. Modifiez le nom complet d'un même auteur dans les deux sessions (avec des valeurs différentes). Que se passe-t-il ?
8. Faites un COMMIT sur la session qui a fait les modifications en premier. Que se passe-t-il ? Faites un select dans les 2 sessions pour voir la modification.
9. Faites un COMMIT dans la deuxième session. Faites un select dans les 2 sessions pour voir la modification.
10. Utilisez un SELECT FOR UPDATE sur une des sessions et essayez de modifier les lignes bloquées avec l'autre session.

EXERCICE 3

1. Ouvrez une nouvelle transaction en "READ ONLY". ***BEGIN TRANSACTION READ ONLY.***
2. Ouvrez en parallèle une deuxième transaction dans laquelle vous modifiez des dates des représentations.
3. Validez cette deuxième transaction. Voyez-vous les modifications dans la première transaction ?
4. Essayez de modifier des données dans la première transaction.
5. Que se serait-il passé si la première transaction n'avait pas été en "READ ONLY" ? Vérifiez-le après avoir terminé la première transaction.

EXERCICE 4

Essayez de faire cet exercice en devinant ce qui va se passer avant de lancer chaque commande.

1. Ouvrez 2 sessions de travail avec des transactions T1 et T2.
2. Passez T1 en mode sérialisé. ***SET TRANSACTION ISOLATION LEVEL SERIALIZABLE.***
3. T1 affiche toutes les représentations.
4. T2 modifie la date de la représentation numéro 150.
5. T1 affiche à nouveau toutes les représentations. Quelle date voit-il pour la représentation 150 ? Pourquoi ?
6. T2 valide sa transaction.
7. T1 affiche à nouveau toutes les représentations. Quelle date voit-il pour la représentation 150 ? Pourquoi ?
8. T1 modifie la date de la représentation numéro 150. Que se passe-t-il ? Pourquoi ?
9. T1 modifie la date d'une autre représentation. Que se passe-t-il ?

Voyez-vous une différence avec le mode par défaut de PostgreSQL ? Explications ?

EXERCICE 5

En travaillant sur la table *LIEU*, vérifiez que dans le mode par défaut de PostgreSQL :

1. Les lectures ne bloquent ni les autres lectures ni les écritures.
2. Les lectures ne sont bloquées par rien, même pas par un blocage d'une table en mode exclusif.
3. Il n'y pas de lecture impropre.
4. Il n'y a pas de pertes de mises à jour (ajoutez 2 fois 10 places à un lieu dans 2 transactions séparées).

EXERCICE 6

Ouvrez 2 sessions et provoquez un interblocage en commençant par faire des modifications sur les tables. Que se passe-t-il ?

EXERCICE 7

Donner un ordonnancement de deux transactions (simples) qui permet, lors de l'exécution, de différencier les modes d'isolation des transactions (read committed ou serializable) en décrivant les résultats attendus pour chacun des modes.